GRANDE ÉCOLE D'INGÉNIEURS

Rapport Technique : Décodeur Morse

25 mars 2025

ALABEATRIX Axel
CHEIKH Hatim
MUSART Jules

Sommaire

1 Introduction

Ce projet vise à concevoir un décodeur Morse interactif sur FPGA Basys 3. Le système doit permettre de convertir des impulsions manuelles en lettres et chiffres alphanumériques, affichés en temps réel sur les LED 7 segments.

L'objectif principal est de développer un système fiable, rapide et ergonomique qui répond aux besoins d'un utilisateur souhaitant convertir des messages en code Morse.

1.1 Cahier des Charges

- Détecter les impulsions de type "point" et "trait" générées par les boutons poussoirs gauche et droit.
- Identifier les caractères alphanumériques correspondant au code Morse entré.
- Afficher les caractères décodés sur les LED 7 segments.

2 Démarche et ètapes de développement

2.1 Analyse des besoins et spécifications

- **Bouton gauche** : Correspond au temps d'un point en code Morse. Lorsqu'appuyé, il est interprété comme un "1".
- **Bouton droit** : Correspond à une pause de la durée d'un point en code Morse. Lorsqu'appuyé, il est interprété comme un "0".
- Trois impulsions du bouton gauche à un trait (—).
- Trois impulsions du bouton droit représentent une pause longue (espace entre deux lettres).

3 Architectures principales

3.1 morse_to_binary

Cette architecture analyse les séquences d'appuis sur les boutons et construit un vecteur binaire représentant le code Morse de chaque lettre :

- **1** : Pour chaque impulsion du bouton gauche.
- **0** : Pour chaque impulsion du bouton droit.

Fonctionnement :

- Lorsque la lettre est terminée (les appuis complets ont été analysés), l'architecture génère un signal `active_o` pour indiquer que le code Morse est prêt à être lu.
- Envoie le vecteur Morse à l'architecture suivante.

3.2 binary_to_ascii

Cette architecture convertit le vecteur binaire Morse en un code ASCII correspondant au symbole Morse. Fonctionnement :

- Une fois le code ASCII généré, l'architecture signale, avec un autre signal, qu'une nouvelle lettre est prête à être affichée.
- Transmet le code ASCII à l'architecture suivante.

3.3 ascii_to_display

Cette architecture gère un afficheur 7 segments capable d'afficher jusqu'à quatre caractères. Fonctionnement :

- Reçoit un code ASCII et affiche la lettre correspondante sur l'afficheur 7 segments.
- Lorsqu'un signal indiquant une nouvelle lettre est reçu, l'afficheur décale les lettres existantes vers la gauche.
- Si l'afficheur est plein (4 caractères), la lettre la plus à gauche disparaît.

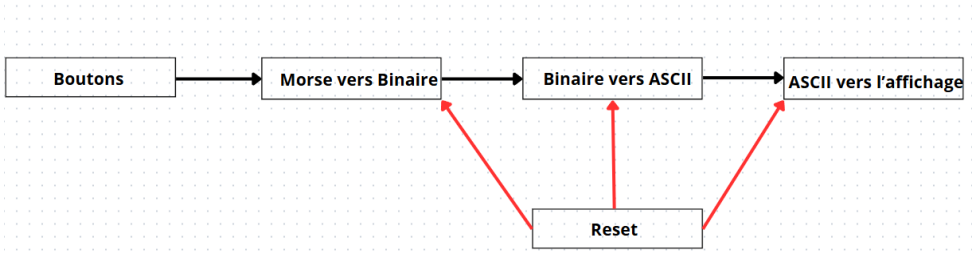
3.4 morse_decoder_top

Cette architecture principale sert à interconnecter les différents modules (extttmorse_to_binary, extttbinary_to_ascii, extttascii_to_display) en définissant clairement leurs relations :

- La sortie de `morse_to_binary` (vecteur binaire Morse et signal `active_o`) est connectée à l'entrée de `binary_to_ascii`.
- La sortie de `binary_to_ascii` (code ASCII et signal de nouvelle lettre) est connectée à l'entrée de `ascii_to_display`.
- Elle gère également les signaux globaux nécessaires à la synchronisation entre les modules, garantissant un flux continu et cohérent des données.

4 Documentation technique

4.1 Synoptique du système



4.2 Chronogramme des impulsions

- Appui court (bouton gauche) = Point (1).
- Appui long (trois fois bouton gauche) = Trait (-).
- Pause courte (bouton droit) = Pause entre signaux d'un même caractère (0).
- Pause longue (trois fois bouton droit) = Pause entre deux lettres.

Le chronogramme des impulsions du projet est représenté dans la figure ci-dessous. Il illustre les différentes étapes de génération des signaux Morse et leurs pauses associées.

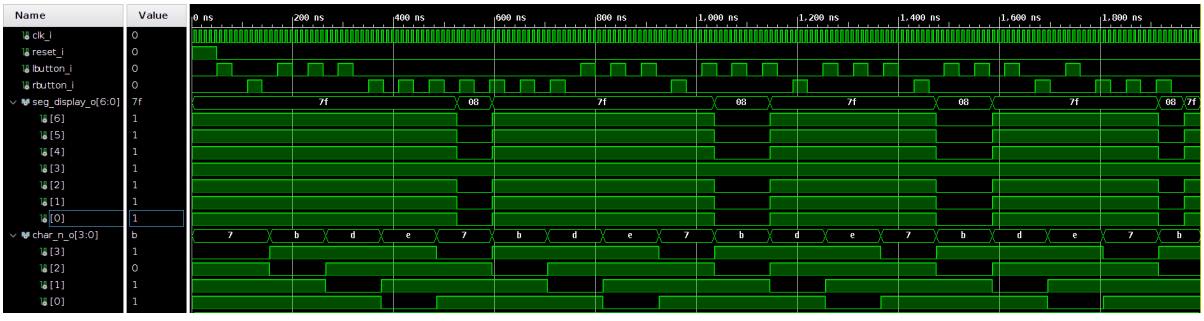


Figure 1. Chronogramme des impulsions du projet de décodeur Morse

Comme indiqué ci-dessus, chaque impulsion correspond à un signal spécifique (point, trait, ou pause).

4.3 Fonctionnement de l'afficheur 7 segments

- L'afficheur 7 segments peut afficher jusqu'à quatre caractères.
- Lorsqu'une nouvelle lettre est reçue, les caractères déjà présents sont décalés vers la gauche.
- Si l'afficheur est plein, la lettre la plus à gauche cesse d'être affichée.

4.4 Algorithme du projet

L'algorithme développé pour ce projet est présenté dans la figure ci-dessous. Il illustre les différentes étapes de traitement des signaux Morse, de la détection à l'affichage :

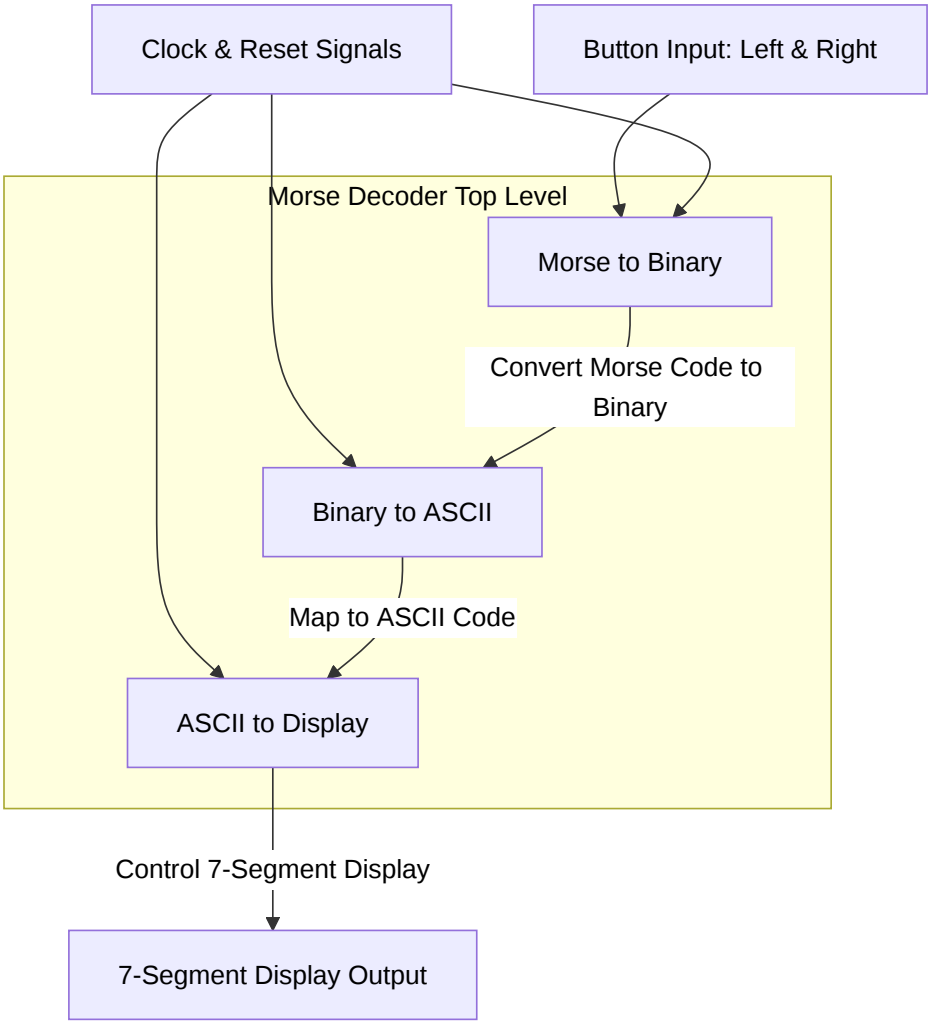


Figure 2. Algorithme du projet de décodeur Morse

Comme illustré dans la figure ci-dessus, les étapes sont interconnectées pour garantir un traitement fluide des données.

5 Fiche d'Utilisation : Application de Décodeur Morse

5.1 Matériel Utilisé

- Carte FPGA : Basys 3
- Bouton d'appui : pour générer les points et les traits.
- Bouton de relâchement : pour signaler les espaces entre lettres et mots.
- Bouton de réinitialisation : pour remettre à zéro la saisie en cours.

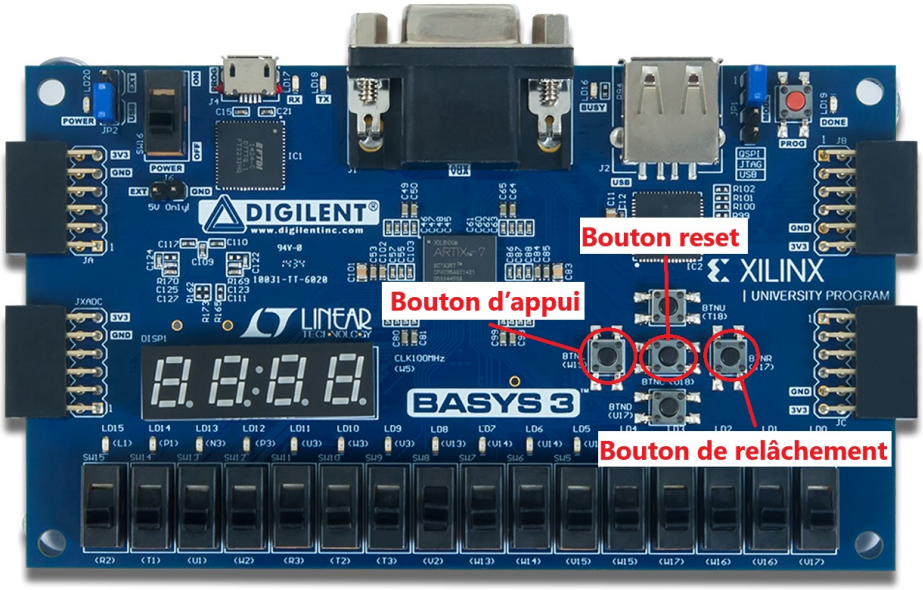


Figure 3. Schéma du fonctionnement du décodeur Morse

5.2 Fonctionnement Détaillé

1. **Génération des Signaux Morse**
   - Point (.) : Appuyez 1 fois sur le bouton d'appui.
   - Trait (-) : Appuyez 3 fois consécutivement sur le bouton d'appui.
2. **Espaces et Transitions**
   - Fin d'un signal (point ou trait) : Appuyez 1 fois sur le bouton de relâchement.
   - Espace entre deux lettres : Appuyez 3 fois sur le bouton de relâchement.
   - Espace entre deux mots : Appuyez 7 fois sur le bouton de relâchement.
   - Affichage de la dernière lettre : Appuyez 3 fois sur le bouton de relâchement.

5.3 Exemple de Codage

Lettre "A" Pour coder un "A" (.-) :

- Appuyez 1 fois sur le bouton d'appui (point).
- Appuyez 1 fois sur le bouton de relâchement (fin du signal).
- Appuyez 3 fois sur le bouton d'appui (trait).

Mot "AB" Pour coder "AB" :

1. Lettre "A" :
   - Appuyez 1 fois sur le bouton d'appui (point).
   - Appuyez 1 fois sur le bouton de relâchement (fin du signal).

- Appuyez 3 fois sur le bouton d'appui (trait).

2. Espace entre lettres :
   - Appuyez 3 fois sur le bouton de relâchement.
3. Lettre "B" :
   - Appuyez 3 fois sur le bouton d'appui (trait).
   - Appuyez 1 fois sur le bouton de relâchement (fin du signal).
   - Appuyez 1 fois sur le bouton d'appui (point).
   - Appuyez 1 fois sur le bouton de relâchement (fin du signal).
   - Appuyez 1 fois sur le bouton d'appui (point).
   - Appuyez 1 fois sur le bouton de relâchement (fin du signal).
   - Appuyez 1 fois sur le bouton d'appui (point).

5.4 Règles Importantes

- Les appuis doivent être clairs et consécutifs pour éviter toute ambiguïté.
- Le compteur se réinitialise automatiquement après un espace (lettre ou mot).
- Assurez-vous de ne pas appuyer simultanément sur les deux boutons pour éviter des erreurs de codage.
- Le bouton de réinitialisation permet d'annuler une saisie incorrecte et de reprendre la saisie depuis le début.

Cette application est une implémentation simple et intuitive pour coder en Morse sans prendre en compte la durée des signaux. Elle permet une manipulation facile des boutons pour créer des points, des traits et des espaces entre lettres et mots. Bonne utilisation !

Légende

- 🔵 : Bouton d'appui (1)
- 🔴 : Bouton de relâchement (0)

Morse Code Sequences

LETTRES :

A: 🔵🔴🔵🔵

B: 🔵🔵🔵🔴🔵🔴🔵🔴🔵

C: 🔵🔵🔵🔴🔵🔴🔵🔵🔵🔴🔵

D: 🔵🔵🔵🔴🔵🔵🔴🔵

E: 🔵

F: 🔵🔴🔵🔵🔴🔵🔵🔴🔵

G: 🔵🔵🔵🔴🔵🔵🔵🔴🔵

H: 🔵🔴🔵🔴🔵🔵🔴🔵

I: 🔵🔴🔵

J: 🔵🔴🔵🔵🔵🔴🔵🔵🔵🔴🔵🔵🔵

K: 🔵🔵🔵🔴🔵🔵🔴🔵🔵🔵

L: 🔵🔴🔵🔵🔵🔴🔵🔵🔴🔵

M: 🔵🔵🔵🔴🔵🔵🔵

N: 🔵🔵🔵🔴🔵

O: 🔵🔵🔵🔴🔵🔵🔵🔴🔵🔵🔵

P: 🔵🔴🔵🔵🔵🔴🔵🔵🔵🔴🔵

Q: 🔵🔵🔵🔴🔵🔵🔵🔴🔵🔴🔵🔵🔵

R: 🔵🔴🔵🔵🔵🔴🔵🔵

S: 🔵🔴🔵🔴🔵🔵

T: 🔵🔵🔵

U: 🔵🔴🔵🔴🔵🔵🔵

V: 🔵🔴🔵🔵🔵🔴🔵🔵🔵

W: 🔵🔴🔵🔵🔵🔴🔵🔵🔵

X: 🔵🔵🔵🔴🔵🔵🔴🔵🔵🔵

Y: 🔵🔵🔵🔴🔵🔵🔵🔵🔴🔵🔵🔵

Z: 🔵🔵🔵🔴🔵🔵🔵🔴🔵🔴🔵🔵

CHIFFRES :

0: 🔵🔵🔵🔴🔵🔵🔵🔴🔵🔵🔵🔴🔵🔵🔵🔴🔵🔵🔵

1: 🔵🔴🔵🔵🔵🔴🔵🔵🔵🔴🔵🔵🔵🔴🔵🔵🔵

2: 🔵🔴🔵🔴🔵🔵🔵🔴🔵🔵🔵🔴🔵🔵🔵

3: 🔵🔴🔵🔴🔵🔴🔵🔵🔵🔴🔵🔵🔵

4: 🔵🔴🔵🔵🔵🔴🔵🔴🔵🔵🔵

5: 🔵🔴🔵🔴🔵🔵🔵🔴🔵

6: 🔵🔵🔵🔴🔵🔵🔵🔴🔵🔴🔵

7: 🔵🔵🔵🔴🔵🔵🔵🔴🔵🔴🔵🔵🔵

8: 🔵🔵🔵🔴🔵🔵🔵🔴🔵🔵🔵🔴🔵🔵🔵

9: 🔵🔵🔵🔴🔵🔵🔵🔴🔵🔵🔵🔴🔵🔵🔵🔴🔵

Espace entre lettres: 🔴🔴🔴

Espace entre mots: 🔴🔴🔴🔴🔴🔴

6 Tests réalisés et résultats

6.1 Tests pratiques

- Test 1 : Détection d'un point et affichage correct sur l'afficheur.
- Test 2 : Détection d'un trait et affichage correct sur l'afficheur.
- Test 3 : Gestion des espaces entre les lettres.
- Test 4 : Gestion du décalage sur l'afficheur 7 segments.

6.2 Résultats observés

- Le système répond en temps réel.
- Les caractères sont correctement identifiés et affichés.
- Les erreurs sont gérées et signalées.

7 Mini-conclusion

7.1 Milestones atteintes vs objectifs

- Détection fiable des points et traits.
- Affichage correct des caractères décodés.

7.2 Ce que nous avons appris

- Compréhension approfondie de la FSM et de son utilisation pour le décodage.
- Optimisation de l'utilisation des ressources FPGA.
- Importance des tests et simulations pour valider un système.

7.3 Points à améliorer

- Améliorer l'interface utilisateur pour une meilleure ergonomie.
- Optimiser les timings pour une détection encore plus précise.